Engineering Research
Express

INCLUSIVE PUBLISHING
TRUSTED SCIENCE

PURPOSE-LED
PUBLISHING™

PAPER

RAD-YOLO: a lightweight edge-deployed visual measurement method for small-object detection in UAV imagery

To cite this article: Shuai Nie *et al* 2026 *Eng. Res. Express* **8** 195204

View the article online for updates and enhancements.

You may also like

- Enhancing small object detection in UAV aerial imagery through integration of global edge information and multi-scale feature enhancement
Hui Chang, Yuru Long, Yiwen Guo et al.
- An improved real-time detection model for small targets in UAV aerial photography based on YOLOv11s
Yan Gao, Min Zhang, Chengyang Tian et al.
- RMD-YOLO11n a small object detection and multi-scale fusion optimization method for UAV images
Jianchao Zhao and Lieping Zhang

This content was downloaded from IP address 183.6.9.92 on 16/10/2026 at 03:40

Engineering Research Express

Check for updates

PAPER

RECEIVED
10 July 2026

REVISED
2 September 2026

ACCEPTED FOR PUBLICATION
16 September 2026

PUBLISHED
30 September 2026

RAD-YOLO: a lightweight edge-deployed visual measurement method for small-object detection in UAV imagery

Shuai Nie, Jiajian Yang, XIN He, Yuhua Qi and Yanqing Hou*

School of Systems Science and Engineering, Sun Yat-sen University (SYSU), Guangzhou, People's Republic of China

* Author to whom any correspondence should be addressed.

E-mail: houyq9@mail.sysu.edu.cn

Keywords: UAV remote sensing, small-object detection, edge deployment

Abstract

Small-object detection in unmanned aerial vehicle (UAV) remote-sensing imagery remains difficult because targets often have low spatial resolution, dense spatial distribution, partial occlusion and strong background clutter. These factors weaken discriminative features and restrict real-time inference on embedded edge platforms. To address the coupled requirements of detection accuracy and deployability, we propose RAD-YOLO, a YOLOv8s-based small-object detector, and develops an edge-deployable variant named RAD-YOLO-Slim. An adaptive-field convolutional block attention module is first introduced into the backbone to enhance multi-scale receptive-field modelling and joint spatial-channel representation. A gated bidirectional asymptotic feature pyramid network is then designed to improve cross-scale semantic interaction while suppressing the propagation of shallow background responses. A distance-normalised Wise-IoU (WIoU) loss and soft non-maximum suppression are further combined to refine bounding-box localisation and retain adjacent detections in dense small-object scenes. Finally, group-shuffle convolution reconstruction, layer-adaptive magnitude-based pruning and 8-bit integer (INT8) post-training quantisation are used to construct RAD-YOLO-Slim for resource-constrained edge deployment. Experiments on VisDrone2019-DET show that RAD-YOLO achieves 47.6% mean average precision at an IoU threshold of 0.5 (mAP@0.5), an improvement of 8.6 percentage points over YOLOv8s under the same protocol. RAD-YOLO-Slim maintains 44.1% mAP@0.5 with 5.42 M parameters and 6.2 giga floating-point operations (GFLOPs). After INT8 deployment on an RK3576 platform, RAD-YOLO-Slim achieves 43.1% mAP@0.5, 51 frames per second (FPS) and 11.81 FPS/W. On the RK3576 platform, RAD-YOLO-Slim provides a practical balance of accuracy, speed and energy efficiency.

1. Introduction

Unmanned aerial vehicle (UAV) remote sensing offers flexible, high-resolution and rapidly acquired observations for traffic monitoring, emergency response, urban management, agricultural inspection and infrastructure surveillance (Wang *et al* 2021, Tong and Wu 2024, Wu and Tong 2025). In these applications, object detection is used for localisation, counting, warning and subsequent tracking. Images are often acquired from elevated or oblique viewpoints. Pedestrians, vehicles and non-motorised vehicles may occupy only a few pixels, while roads, roofs, vegetation, shadows and building textures create complex backgrounds. The task is thus to preserve weak target cues while distinguishing them from background responses.

Deep-learning detectors are generally grouped into two-stage and one-stage methods. Two-stage detectors, represented by Faster R-CNN, first generate candidate regions and then perform classification and bounding-box regression. This design can provide reliable localisation but increases computational cost (Ren *et al* 2015). One-stage detectors, including SSD, RetinaNet, CenterNet and the YOLO

© 2026 IOP Publishing Ltd. All rights, including for text and data mining, AI training, and similar technologies, are reserved.

series, directly predict class labels and bounding boxes from dense feature maps, making them suitable for real-time UAV perception (Liu *et al* 2016, Redmon *et al* 2016, Lin *et al* 2017, Zhou *et al* 2019). YOLOv8 serves as the baseline here because it provides an efficient one-stage structure, anchor-free prediction and a practical speed-accuracy trade-off (Jocher *et al* 2023).

Recent UAV-oriented detectors have sought to address these difficulties with attention mechanisms, multi-scale feature fusion, high-resolution detection heads and lightweight operators (Zhu *et al* 2021, Wang *et al* 2023, Zhang 2023, Liu *et al* 2024, Yue *et al* 2024, Zhang *et al* 2024, Dong *et al* 2025). Three practical issues remain relevant to this problem. First, repeated downsampling can remove weak geometric and texture cues before they reach the detection head. Second, shallow feature maps preserve spatial details but also contain abundant background responses that may be amplified during feature-pyramid fusion. Third, bounding-box regression and hard non-maximum suppression (NMS) can be less stable when many small objects are close to one another. These challenges make distant, partially occluded and densely clustered objects difficult to detect in UAV imagery.

Deployment imposes additional constraints. UAV platforms may have limited power, memory, heat-dissipation capacity and payload capacity. Detectors optimised on desktop graphics processing unit (GPUs) may consequently be difficult to run in real time on board. Lightweight convolution, structured pruning and 8-bit integer (INT8) quantisation can lower parameter count and computation, but excessive compression may damage the shallow detail pathways that are important for small-object detection (Howard *et al* 2017, Jacob *et al* 2018, Zhang *et al* 2018, Lee *et al* 2021). A detector intended for onboard use therefore needs a pipeline-level design: feature enhancement should improve detectability, fusion should control background interference, localisation and post-processing should handle dense targets, and compression should preserve useful feature pathways.

To address these challenges, we propose RAD-YOLO and its lightweight variant, RAD-YOLO-Slim. Our main contributions are as follows: (1) An adaptive-field convolutional block attention module (AFCBAM) is designed to improve small-object representation through adaptive multi-scale receptive fields and spatial-channel attention. (2) A gated bidirectional asymptotic feature pyramid network (G-BiAFPN) is developed for selective bidirectional feature fusion, thereby reducing background interference during cross-scale interaction. (3) A distance-normalised wise-intersection over union (IoU) loss (D-WIoU) and Soft-NMS are combined to improve localisation sensitivity and reduce over-suppression among neighbouring small objects. (4) A lightweight edge-deployment workflow based on group-shuffle convolution (GSConv), layer-adaptive magnitude-based pruning (LAMP) and INT8 quantisation is implemented on RK3576. For RAD-YOLO-Slim, mAP@0.5, mAP@0.5:0.95, inference speed and energy efficiency are reported. For RAD-YOLO, we also report precision and recall.

The paper is organised as follows. Section 2 reviews related work and identifies the research gap. Section 3 describes the RAD-YOLO architecture and its core modules. Section 4 presents the lightweight design and edge-deployment workflow. Section 5 reports the experimental setup, ablation studies, comparative evaluation and RK3576 deployment results. Section 6 discusses the accuracy-efficiency trade-off, limitations and directions for further study, and section 7 concludes the paper.

2. Related work and research gap

Object detection in UAV imagery has progressed from general-purpose frameworks toward architectures that account for viewpoint, scale and object density. Classical two-stage detectors improve localisation by using region proposals, but their multi-stage inference is often expensive for real-time edge applications (Ren *et al* 2015). One-stage detectors reduce this cost by predicting classes and bounding boxes directly from feature maps. SSD, RetinaNet and CenterNet represent different strategies for dense prediction and keypoint-based localisation, while YOLO-series models are widely used because of their speed and engineering maturity (Liu *et al* 2016, Redmon *et al* 2016, Lin *et al* 2017, Zhou *et al* 2019, Ge *et al* 2021, Wang *et al* 2024).

Many methods improve small-object feature representation by increasing scale diversity, preserving high-resolution information or adding local context. Attention modules such as SE and convolutional block attention module (CBAM) recalibrate feature responses in channel and spatial dimensions, helping detectors focus on informative regions under cluttered backgrounds (Hu *et al* 2018, Woo *et al* 2018). Feature pyramid structures, including adaptive spatial feature fusion, bidirectional feature pyramid network and attention-based SSD fusion, improve multi-scale information exchange, but they may also transmit shallow background texture if feature selection is weak (Liu *et al* 2019, Tan *et al* 2020, Yin *et al* 2020). This line of work suggests that multi-scale fusion needs to be selective rather than simply additive.

Localisation and post-processing introduce additional errors in dense aerial scenes. Complete-IoU (CIoU), Efficient-IoU and Wise-IoU (WIoU) improve bounding-box regression by considering overlap, centre distance, aspect ratio and sample quality (Zhang *et al* 2022, Tong *et al* 2023). However, for a target occupying only a few pixels, even a small centre displacement can cause a substantial change in the predicted overlap. Depending on the normalisation scheme, the effect of this displacement may not be adequately reflected in the regression loss. During post-processing, hard NMS discards a lower-scoring box when its overlap with a higher-scoring box exceeds a predefined threshold. In crowded scenes, this rule can suppress valid detections of nearby objects. Soft-NMS reduces the confidence score of overlapping boxes instead of removing them directly, allowing valid neighbouring detections to be retained when their adjusted scores remain above the detection threshold (Bodla *et al* 2017).

Lightweight models and deployment strategies are also important in UAV remote sensing. MobileNet and ShuffleNet show that efficient convolution and channel interaction can reduce redundancy, while LAMP pruning and INT8 quantisation provide practical compression and deployment mechanisms (Howard *et al* 2017, Jacob *et al* 2018, Zhang *et al* 2018, Lee *et al* 2021). Compression alone, however, is insufficient: pruning and quantisation need to preserve shallow spatial details and stable cross-scale semantics. The deployment requirement should therefore be considered during model design.

Existing studies address feature enhancement, attention, feature fusion, localisation losses and lightweight design. Here, we integrate and evaluate these techniques in one detection-and-deployment pipeline. The individual techniques have been studied previously. This study combines them in a single UAV small-object detection pipeline and evaluates the resulting model on an edge platform.

3. RAD-YOLO model design and rationale

3.1. YOLOv8s baseline and overall architecture

RAD-YOLO is built on YOLOv8s and retains its one-stage detection pipeline. The architecture was organised around the error sources described above. AFCBAM is placed in the backbone to enhance weak small-object responses before repeated downsampling removes fine details. G-BiAFPN is used in the neck to align shallow geometric details with deeper semantic cues while suppressing background noise. D-WIoU is used during training, whereas Soft-NMS is applied during inference to improve localisation sensitivity and reduce mistaken suppression in dense scenes. As shown in figure 1, the backbone and neck components form the detection architecture, while D-WIoU and Soft-NMS operate during training and inference, respectively.

3.2. AFCBAM feature-enhanced backbone module

AFCBAM is placed in the backbone to strengthen small-object feature representation. Its structure is shown in figure 2. Repeated downsampling can weaken fine-grained target information, particularly when an object occupies only a few pixels. Compared with the original CBAM, which uses fixed spatial attention, AFCBAM introduces three parallel grouped-convolution branches with dilation rates of 1, 2 and 3, respectively. The three branches are intended to capture local texture and edge cues, neighbouring context and broader contextual relations, respectively. Global pooling and softmax gating then assign branch weights according to the image content. The weighting process is given in equation (1). Here, the output on the left denotes the adaptive multi-scale feature, F is the input feature, d_i denotes the dilation rate of the ith branch, g_i is the branch response used for softmax gating, and α_i is the normalised branch weight.

$$F_{\mathrm{ms}} = \sum_{i=1}^{3} \alpha_i \mathrm{Conv}_{d_i}(F), \ \alpha_i = \frac{\exp(g_i)}{\sum_{j=1}^{3} \exp(g_j)} \tag{1}$$

After extracting the multi-scale branches, AFCBAM applies a dimensional transformation followed by joint attention modelling. The transformation redistributes spatial information across channels, which may help separate weak target regions from surrounding texture. The spatial branch highlights potential object locations, while the channel branch emphasises semantically informative responses. The two attention outputs are fused and mapped back to the original feature space, as expressed in equation (2), where $\odot$ denotes element-wise multiplication, M_{s} and M_{c} are the spatial and channel attention maps, and F_{out} is the enhanced feature. Within the proposed architecture, AFCBAM targets the first issue by preserving and strengthening small-object cues before feature fusion.

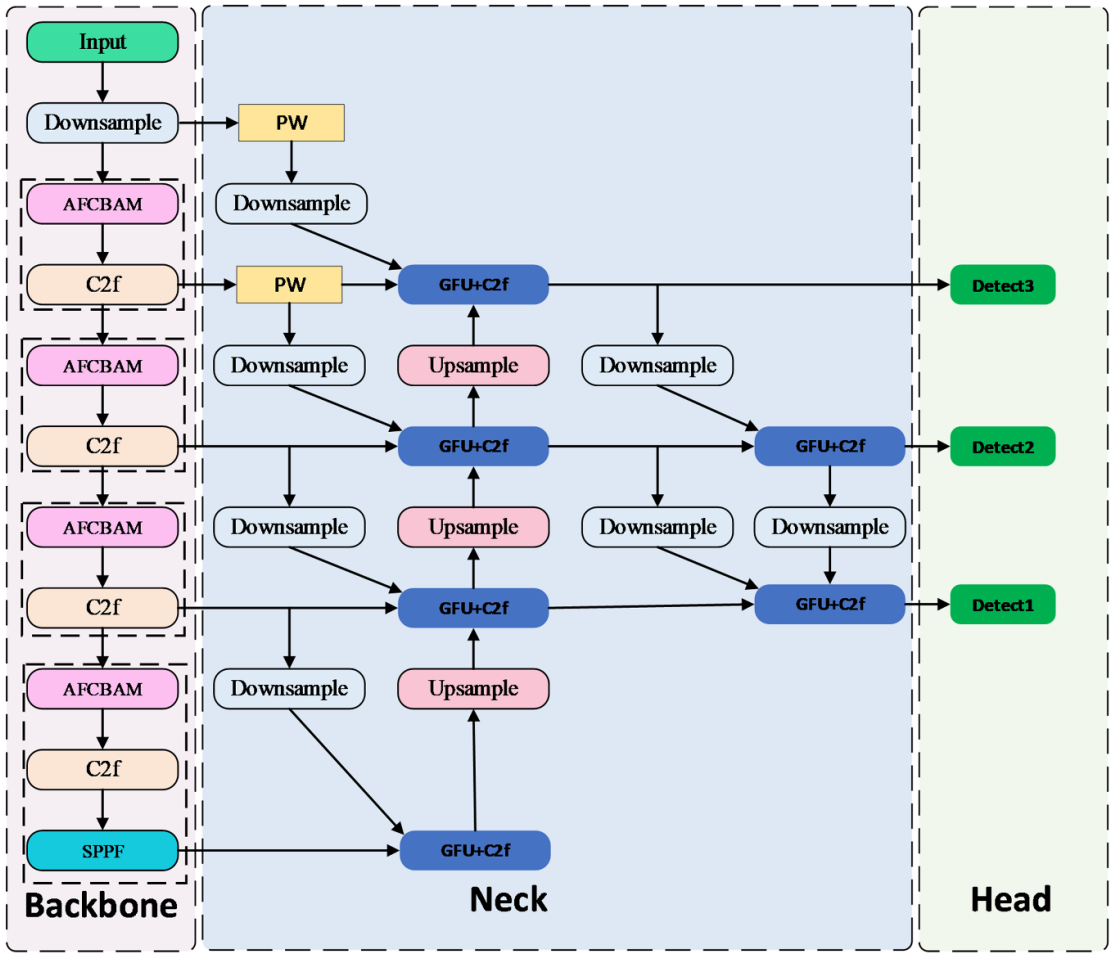


Figure 1. Architecture of the proposed RAD-YOLO network.

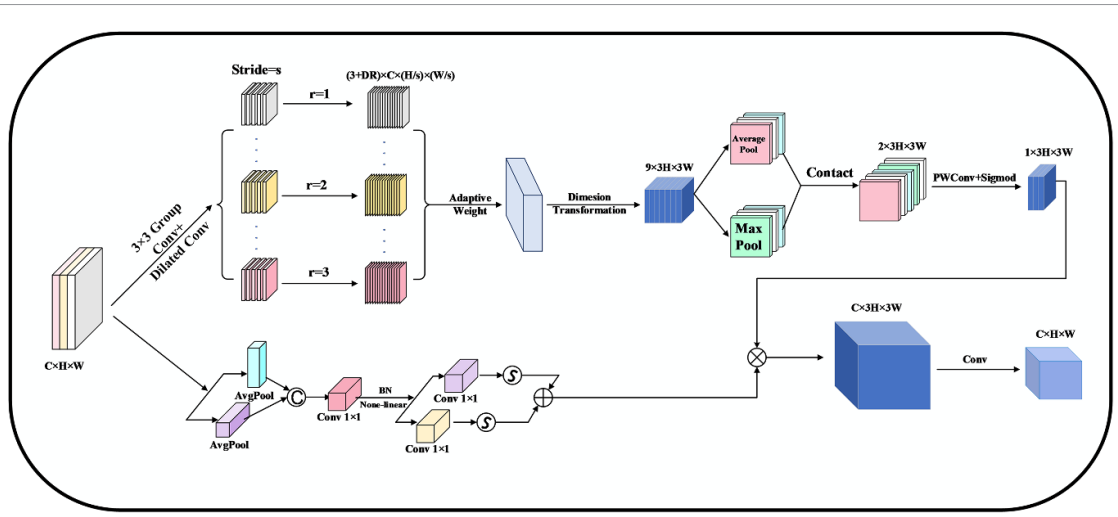


Figure 2. Structure of the AFCBAM module.

$$F_{\text{out}} = F_{\text{ms}} \odot M_{\text{s}}(F_{\text{ms}}) \odot M_{\text{c}}(F_{\text{ms}}) \tag{2}$$

3.3. G-BiAFPN gated cross-scale fusion network

Figure 3(a) and (b) illustrate the structure of the gated fusion unit and the overall G-BiAFPN feature-fusion network, respectively. G-BiAFPN facilitates interaction between shallow and deep features. Small objects depend on shallow high-resolution details, whereas reliable classification requires deeper semantic information. A conventional PAN-FPN structure aggregates features through concatenation or simple

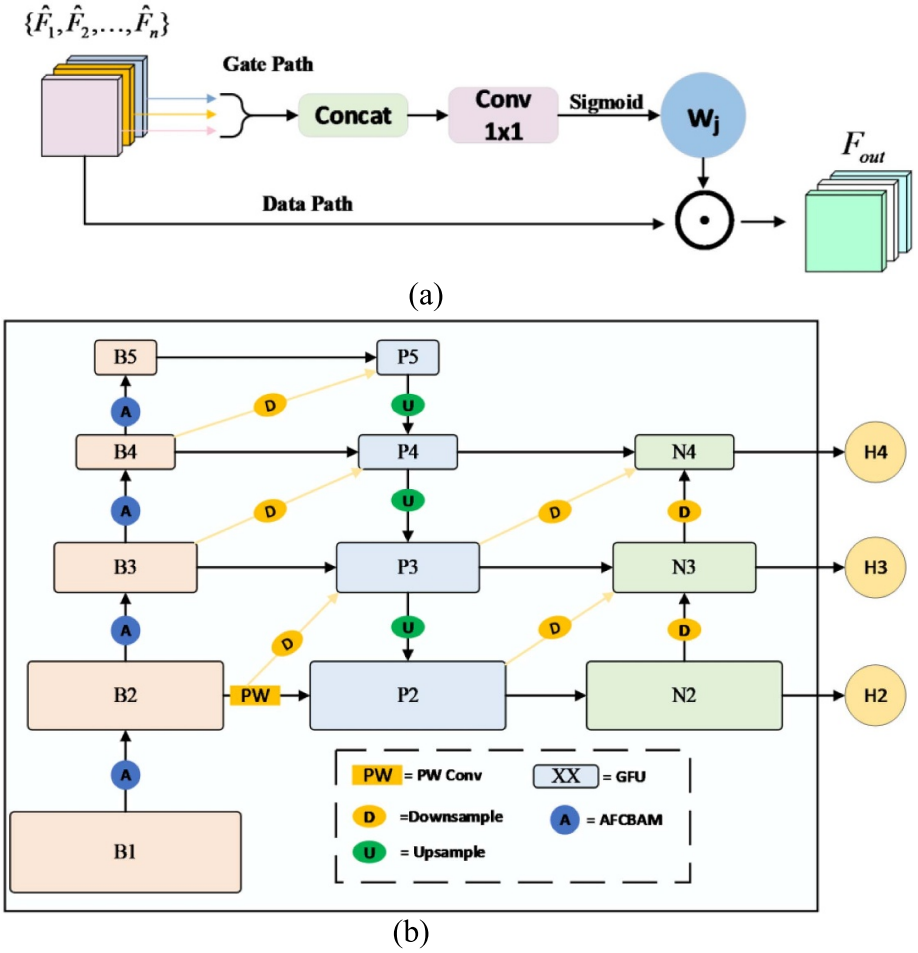


Figure 3. (a) Structure of the gated fusion unit. (b) Structure of the G-BiAFPN feature fusion network.

weighted fusion, but this may transmit road texture, roof patterns, vegetation and shadows together with target information. G-BiAFPN extends fusion to higher-resolution layers and inserts gated fusion units (GFUs) at cross-scale nodes. The top-down path transfers semantic context to fine-resolution features, and the bottom-up path returns localisation details to higher-level features. The gates are intended to increase the contribution of target-related branches and suppress conflicting or noisy responses.

At each fusion node, the GFU concatenates features from different scales and generates content-adaptive weights with a convolution and sigmoid activation. Each input branch is multiplied by its corresponding gate, and the gated features are then summed to form the fused output. Unlike simple concatenation, this operation allows the network to select the dominant scale under different scene conditions. In dense aerial images, the GFU is intended to preserve shallow object boundaries while reducing the influence of irrelevant texture. The fusion process is given in equation (3). In this equation, the left-hand term denotes the fused output, F_k is the kth input feature, β_k is the gating coefficient, and σ denotes the sigmoid function. The summation upper bound is the number of input branches. This gives G-BiAFPN a selective, rather than indiscriminate, cross-scale fusion mechanism. Each gate is sigmoid-constrained, and the upper limit of the summation is the number of input branches at the fusion node.

$$F_o = \sum_{k=1}^{K} \beta_k F_k, \quad \beta = \sigma\left(\mathrm{Conv}\left([F_1, \ldots, F_K]\right)\right) \tag{3}$$

3.4. Dense small-object localisation with D-WIoU and soft-NMS

D-WIoU is designed to mitigate localisation errors associated with the small spatial extent of UAV targets. A small centre displacement can account for a large proportion of the target size. CIoU and WIoU typically normalise the centre-distance term by the scale of the minimum enclosing box. In wide-field UAV imagery, this enclosing region may be much larger than the target, so small but important localisation errors may receive insufficient gradient constraints. D-WIoU modifies this term by normalising the

distance penalty with the ground-truth box scale, which is intended to increase sensitivity to small-target offsets in the regression loss. The formulation appears in equation (4). In this equation, the left-hand term is the proposed D-WIoU loss, the first term on the right is the original Wise-IoU loss, (x, y) and (x^{gt}, y^{gt}) are the predicted and ground-truth centres, w^{gt} and h^{gt} are the target width and height, λ is the distance weight, and ε avoids division by zero.

$$L_{\text{D - WIoU}} = L_{\text{WIoU}} + \lambda \frac{(x - x^{gt})^2 + (y - y^{gt})^2}{(w^{gt})^2 + (h^{gt})^2 + \varepsilon} \tag{4}$$

D-WIoU retains the dynamic focusing mechanism of Wise-IoU, so samples of different quality receive different gradient gains during training. During inference, Soft-NMS is used to reduce the potential over-suppression caused by hard NMS in dense scenes. Rather than deleting boxes that overlap a higher-confidence prediction, Soft-NMS gradually decays their confidence scores. This may be useful when adjacent vehicles, pedestrians or bicycles overlap from the UAV perspective. The update rule is presented in equation (5), where s_i denotes the candidate confidence, b_m is the highest-confidence box, b_i is the candidate box, N_t is the overlap threshold, and σ is the decay coefficient. D-WIoU and Soft-NMS thus target the third issue by improving localisation sensitivity and reducing over-suppression among neighbouring small objects.

$$s_i = \begin{cases} s_i, & \text{IoU}(b_m, b_i) < N_t \\ s_i \exp\left(-\text{IoU}(b_m, b_i)^2/\sigma\right), & \text{IoU}(b_m, b_i) \geq N_t \end{cases} \tag{5}$$

4. RAD-YOLO-slim lightweight design and edge deployment

RAD-YOLO improves feature representation and localisation, but its additional modules increase inference cost. To make the detector suitable for UAV edge computing, the accuracy-oriented model is compressed into RAD-YOLO-Slim. The lightweight design follows a conservative principle: reducing redundant computation while preserving the feature pathways that support small-object detection. GSConv reconstruction first reduces convolutional redundancy in the neck and detection head. LAMP pruning then removes structurally redundant channels while accounting for layer-level sensitivity. Finally, INT8 post-training quantisation adapts the model to the RK3576 neural processing unit (NPU) execution environment.

GSConv combines standard convolution, depthwise separable convolution and channel shuffling. The standard convolution branch provides cross-channel mixing, whereas the depthwise branch reduces convolutional cost. Channel shuffling is used to promote information exchange between channel groups. Here, the structure replaces selected expensive convolutions in feature-fusion modules. Its computational effect is evaluated in table 3. In this study, GSConv is evaluated in the neck and detection head separately, and the final RAD-YOLO-Slim model uses the configuration selected from these tests. The computational comparison in equation (6) reports the standard-convolution cost and the depthwise-separable cost, respectively. In this expression, the leading height and width factors denote spatial resolution, C_{in} and C_{out} are the input and output channels, and k is the kernel size.

$$G_{\text{std}} = HWC_{\text{in}}C_{\text{out}}k^2, \quad G_{\text{dw}} = HW\left(C_{\text{in}}k^2 + C_{\text{in}}C_{\text{out}}\right) \tag{6}$$

LAMP is used for structured pruning because it evaluates weight importance within each layer rather than applying a single global threshold. This distinction matters for UAV small-object detection, where shallow layers and cross-scale fusion channels may have different sensitivity to pruning. LAMP estimates layer-adaptive sparsity from the cumulative magnitude distribution and removes redundant channels under a constrained pruning schedule. Several target acceleration ratios are evaluated. The main lightweight configuration uses a target acceleration ratio of 2.5. Here, the acceleration ratio denotes the target computational acceleration factor relative to the pre-pruned RAD-YOLO-Slim configuration, approximately expressed as the baseline GFLOPs divided by the post-pruning GFLOPs. It should not be confused with the pruning rate, which denotes the proportion of channels removed. Because the pruning rate may differ across layers, a single pruning rate does not correspond to a fixed acceleration ratio across all configurations. The importance score is given in equation (7). In this equation, the left-hand

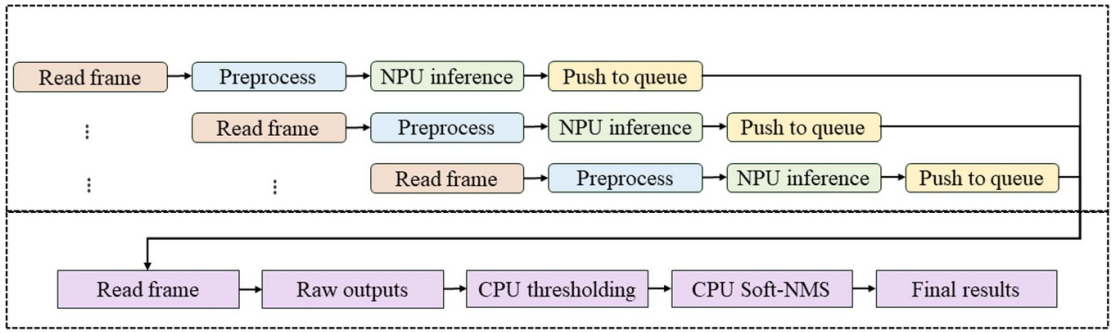


Figure 4. Asynchronous CPU-NPU inference pipeline on the RK3576 edge platform.

term is the LAMP importance score, where $w_{l,i}$ denotes the weight at rank i in the layer; j is the summation index. n_l is the number of weights in that layer, and ε is a numerical-stability term.

$$S_l(i) = \frac{w_{l,i}^2}{\sum_{j=i}^{n_l} w_{l,j}^2 + \varepsilon} \tag{7}$$

The deployment workflow includes open neural network exchange export, RKNN-Toolkit2 conversion and INT8 post-training quantisation. Approximately 200 images from the VisDrone training set are used for calibration to estimate the activation range. 32-bit floating-point (FP32) weights and activations are then mapped to INT8 representations for the RK3576 NPU. Equation (8) gives this mapping, where x is the floating-point value, q is the quantised integer value, s is the scale, z is the zero point, and $q_{\min}$ and $q_{\max}$ define the minimum and maximum integer range, respectively. The reconstructed value on the right-hand side is the dequantised value; round denotes nearest-integer rounding and clip limits the result to the interval defined by these bounds. The post-quantisation evaluation therefore follows the same conversion path used for edge deployment. The scale converts the floating-point interval to the integer interval, the zero point shifts the quantised range, and the two bound variables in equation (8) specify the supported lower and upper limits of the INT8 integer range.

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \quad \hat{x} = s(q - z) \tag{8}$$

The edge implementation uses a heterogeneous central processing unit (CPU)-NPU pipeline. The CPU reads images, performs preprocessing, decodes model outputs and executes Soft-NMS, whereas the NPU performs INT8 forward inference. Asynchronous multithreading allows preprocessing, NPU computation and post-processing to overlap. With ideal overlap, the pipeline period is bounded by the slowest stage rather than the serial sum of all stages. Figure 4 shows the pipeline. Equation (9) expresses the throughput constraint, where T_{pre}, T_{npu} and T_{post} denote preprocessing, NPU inference and post-processing time, and T_{pipe} is the single-frame pipeline period. FPS is reported as frames per second, with the time variables in equation (9) expressed in milliseconds. Equation (9) describes the ideal overlap between the pipeline stages. In practice, the measured end-to-end FPS may include system, memory-transfer and scheduling overhead. It may therefore differ from the value predicted by the ideal expression.

$$T_{\mathrm{pipe}} \approx \max\left(T_{\mathrm{pre}}, T_{\mathrm{npu}}, T_{\mathrm{post}}\right), \quad \mathrm{FPS} = \frac{1000}{T_{\mathrm{pipe}}} \tag{9}$$

5. Experiments and data analysis

5.1. Dataset, experimental environment and evaluation metrics

The evaluation proceeds in four stages: detection accuracy, module effectiveness, lightweight design and edge-deployment feasibility. The experiments use VisDrone2019-DET as the main dataset, a UAV-view object detection benchmark from the VisDrone challenge (Zhu *et al* 2018) containing pedestrians, people, bicycles, cars, vans, trucks and other traffic-related categories. The dataset contains dense instances,

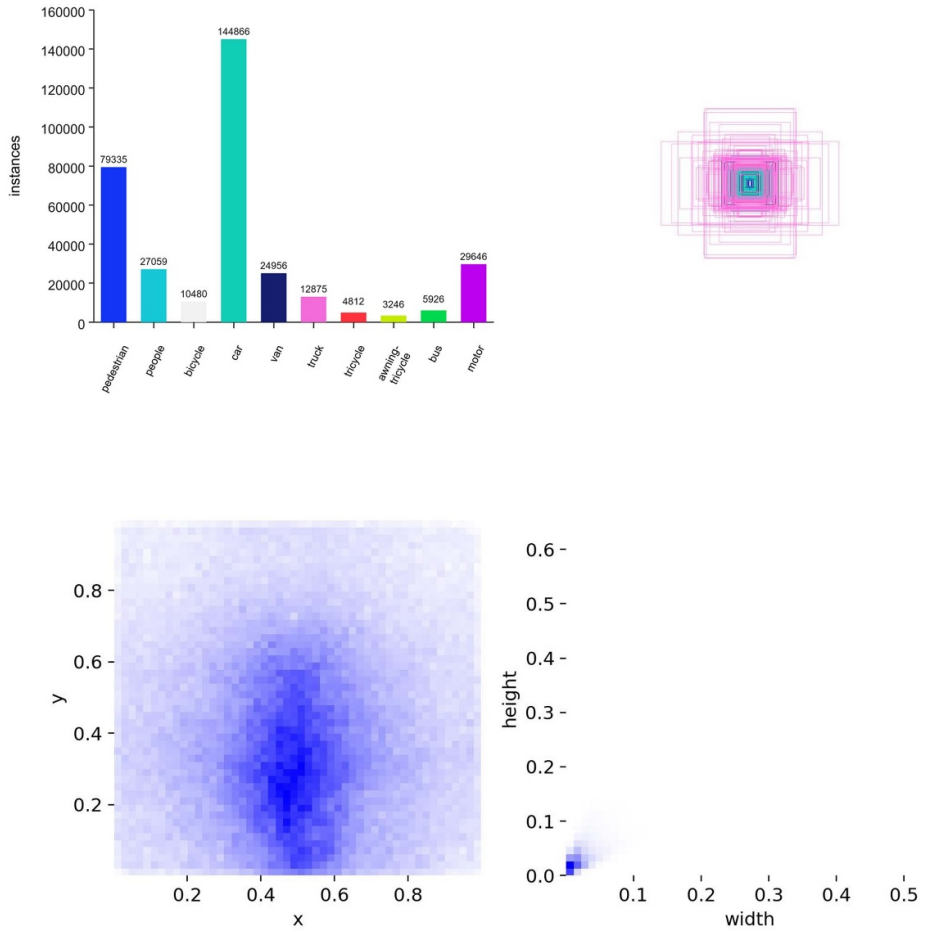


Figure 5. Category and object-scale distributions in VisDrone2019-DET.

complex backgrounds and images captured at different heights, viewpoints and illumination conditions. The server-side training and testing environment includes Ubuntu 22.04.4 long-term support, an Intel Xeon Platinum 8488C CPU, 256 GB random-access memory (RAM), an NVIDIA RTX 4090 GPU, PyTorch 1.13.1, Python 3.8.20 and Compute Unified Device Architecture (CUDA) 12.4.

VisDrone2019-DET contains 6471 training images, 548 validation images and 1610 test images. The category counts in the training set, plotted in figure 5, sum to 343 201 annotated objects. The scale distribution is concentrated in the small-object range. Figure 5 presents the category and object-scale distributions. These distributions provide the context for the small-object evaluation.

Precision measures the proportion of correct detections among predicted objects, while recall measures the proportion of ground-truth objects that are detected. Mean average precision (mAP) at an IoU threshold of 0.5 is denoted as mAP@0.5, and mAP@0.5:0.95 averages AP over IoU thresholds from 0.5 to 0.95. For deployment analysis, NPU inference latency, pipeline FPS, average power, peak power and energy efficiency are also used. The metrics cover detection quality, computational cost and deployment behaviour. Precision and recall are defined in equation (10), where TP, FP and FN represent true positives, false positives and false negatives. Average precision and energy efficiency are defined in equation (11), where C is the number of categories; R denotes recall, $P_c(R)$ denotes precision at recall R, and AP_c denotes category-wise average precision obtained by integration over the precision-recall curve. The final term in equation (11) denotes energy efficiency, and P_{avg} is average power. Energy efficiency is reported as FPS per watt.

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{10}$$

$$\text{AP}_c = \int_0^1 P_c(R)\,\mathrm{d}R, \ \text{mAP} = \frac{1}{C}\sum_{c=1}^{C}\text{AP}_c, \ \eta = \frac{\text{FPS}}{P_{\text{avg}}} \tag{11}$$

Table 1. Progressive ablation results for RAD-YOLO.

Baseline	AFCBAM	G-BiAFPN	D-WIoU	Soft-NMS	Precision (%)	Recall (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)
Yes					50.9	38.1	39.0	22.8
Yes	Yes				54.1	39.8	41.2	24.5
Yes	Yes	Yes			56.7	41.9	44.1	26.2
Yes	Yes	Yes	Yes		57.2	42.8	44.5	26.9
Yes	Yes	Yes	Yes	Yes	58.4	44.2	47.6	28.3

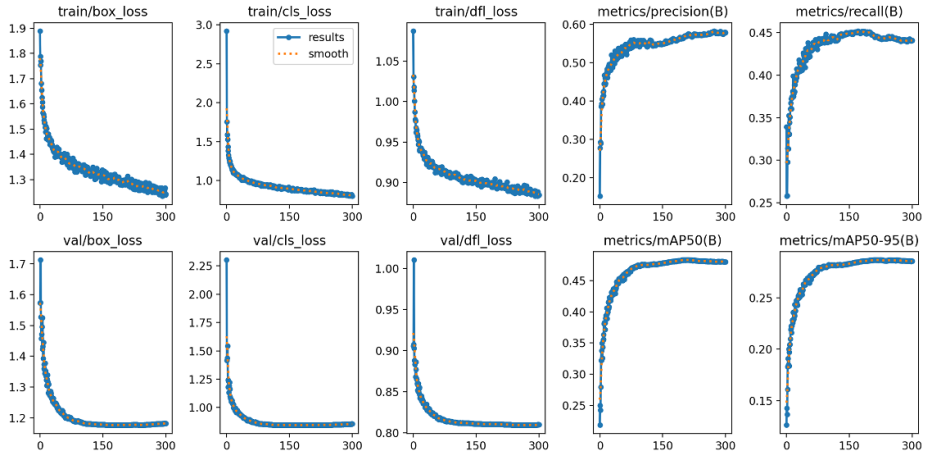


Figure 6. Training convergence curves of RAD-YOLO.

Table 2. Performance comparison with representative detectors.

Model	mAP@0.5 (%)	mAP@0.5:0.95 (%)	GFLOPs	Parameters (M)
Faster R-CNN	37.2	21.9	∼180	41.16
RetinaNet	19.1	10.6	∼151	35.7
CenterNet	33.7	18.8	∼35	70.75
YOLOv5s	37.7	22.3	15.8	9.11
YOLOv8s	39.0	22.8	28.8	11.1
YOLOv11s	39.3	23.6	21.5	9.41
RAD-YOLO	47.6	28.3	28.5	13.5
RAD-YOLO-Slim	44.1	24.5	6.2	5.42

5.2. Ablation studies and high-accuracy model performance

The progressive results in table 1 provide evidence of complementary contributions from the proposed components. Figure 6 shows the training convergence curves of RAD-YOLO. The progressive results in table 1 provide evidence of complementary contributions from the proposed components. Starting from YOLOv8s, adding AFCBAM raises mAP@0.5% from 39.0% to 41.2%. Adding G-BiAFPN further increases it to 44.1%. With D-WIoU, the score reaches 44.5%, and the full RAD-YOLO with Soft-NMS achieves 47.6% mAP@0.5 and 28.3% mAP@0.5:0.95. Precision and recall reach 58.4% and 44.2%, respectively. The 8.6-percentageimprovement over YOLOv8s is consistent with complementary contributions from enhanced backbone features, selective fusion and dense-object post-processing.

Table 2 presents a comparison between RAD-YOLO and representative detectors. Faster R-CNN, RetinaNet and CenterNet achieve 37.2%, 19.1% and 33.7% mAP@0.5, respectively. Among YOLO-style baselines, YOLOv5s, YOLOv8s and YOLOv11s obtain mAP@0.5 values of 37.7%–39.3%. RAD-YOLO reaches 47.6% mAP@0.5 with 13.5 M parameters and 28.5 GFLOPs, higher than the values listed for the comparison models. The FP32 RAD-YOLO-Slim configuration reaches 44.1% mAP@0.5 with 6.2 GFLOPs before INT8 deployment, indicating that the lightweight version retains much of the accuracy advantage of the full model.

The cited UAV-oriented studies use lightweight feature extraction, UAV-specific modelling or efficient deployment (Wang *et al* 2023, Zhang 2023, Yue *et al* 2024). Because these methods are not included as

Table 3. Ablation results for GSConv reconstruction.

Experiment no.	Model	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Parameters (M)	GFLOPs	FPS
1	RAD-YOLO	47.6	28.3	13.5	28.5	85
2	+Neck(GSConv)	47.1	27.9	10.25	19.8	108
3	+Head(GSConv)	47.2	27.9	12.3	24.2	96
4	RAD-YOLO-Slim (before pruning)	46.9	27.8	9.2	15.5	121

Table 4. Performance under different LAMP pruning acceleration ratios.

Acceleration ratio	mAP@0.5 (%)	mAP@0.5:0.95 (%)	GFLOPs	FPS	Parameters (M)
1.0	46.9	27.8	15.5	121	9.20
1.5	46.5	27.4	10.3	145	7.12
2.0	45.4	26.3	7.8	162	6.05
2.5	44.1	24.5	6.2	175	5.42
3.0	42.1	22.6	5.1	188	4.35

Table 5. INT8 deployment performance on the RK3576 platform.

Model	mAP@0.5 (%)	NPU inference latency (ms)	FPS
YOLOv8s	38.2	28.5	35
RAD-YOLO	46.5	35.4	22
RAD-YOLO-Slim	43.1	18.8	51

rows in table 2, the present study does not make a direct numerical comparison with them. The comparison in table 2 is limited to the listed detectors and should be interpreted within the evaluation setting reported there. RAD-YOLO-Slim is the deployment-oriented variant evaluated in this study.

5.3. Lightweight design and pruning analysis

Table 3 shows the effect of GSConv reconstruction. The original RAD-YOLO achieves 47.6% mAP@0.5 with 13.5 M parameters and 28.5 GFLOPs. Replacing selected convolutions in the neck or head reduces computation and increases FPS, with a limited decrease in detection accuracy. The GSConv-reconstructed FP32 configuration before LAMP pruning reaches 46.9% mAP@0.5, 9.2 M parameters and 15.5 GFLOPs. The lower parameter count and GFLOPs accompany the speed increase after replacing selected convolutions while retaining sufficient feature interaction for small-object detection.

Table 4 shows the effect of different LAMP pruning acceleration ratios. Increasing the ratio reduces GFLOPs and parameter count and increases FPS, but detection accuracy gradually declines. At a 1.5 ratio, the model achieves 46.5% mAP@0.5 and 10.3 GFLOPs. At a 2.0 ratio, it achieves 45.4% mAP@0.5 and 7.8 GFLOPs. Before INT8 deployment, the model at a 2.5 ratio reaches 44.1% mAP@0.5 with 6.2 GFLOPs and 5.42 M parameters. At a 3.0 ratio, accuracy drops to 42.1%. The 2.5 ratio is used for the lightweight configuration because it provides a favourable compression benefit while preserving a 5.1-percentage-point mAP@0.5 advantage over YOLOv8s. This choice reflects the trade-off among accuracy, computation and speed.

5.4. RK3576 deployment, power evaluation and qualitative real-scene assessment

The RK3576 experiment tests the lightweight model after conversion and quantisation. For RAD-YOLO-Slim, CPU preprocessing takes approximately 10.5 ms per frame, NPU forward inference takes 18.8 ms, and CPU decoding with Soft-NMS takes 14.2 ms. If these stages are executed serially, the total processing time reaches 43.5 ms. The reported end-to-end throughput uses the asynchronous CPU-NPU pipeline. This whole-pipeline measure complements isolated model latency when describing a deployed UAV perception system.

Table 5 summarises deployment performance after INT8 quantisation. YOLOv8s obtains 38.2% mAP@0.5 and 35 FPS. The full RAD-YOLO achieves 46.5% mAP@0.5 after deployment and is measured at 22 FPS after INT8 deployment on RK3576. RAD-YOLO-Slim obtains 43.1% mAP@0.5, 18.8 ms NPU inference latency and 51 FPS. Compared with RAD-YOLO, the lightweight model reduces mAP@0.5 by 3.4 percentage points and more than doubles the measured frame rate.

Table 6. Accuracy comparison before and after INT8 quantisation.

Model	FP32 mAP@0.5 (%)	INT8 mAP@0.5 (%)	Accuracy loss (percentage points)
YOLOv8s	39.0	38.2	0.8
RAD-YOLO	47.6	46.5	1.1
RAD-YOLO-Slim	44.1	43.1	1.0

Table 7. Power consumption and energy efficiency on the RK3576 platform.

State/model	Average power (W)	Peak power (W)	FPS	Energy efficiency (FPS/W)
Idle system	0.96	5.42	—	—
YOLOv8s	4.25	7.51	35	8.24
RAD-YOLO	5.38	9.13	22	4.09
RAD-YOLO-Slim	4.32	7.95	51	11.81

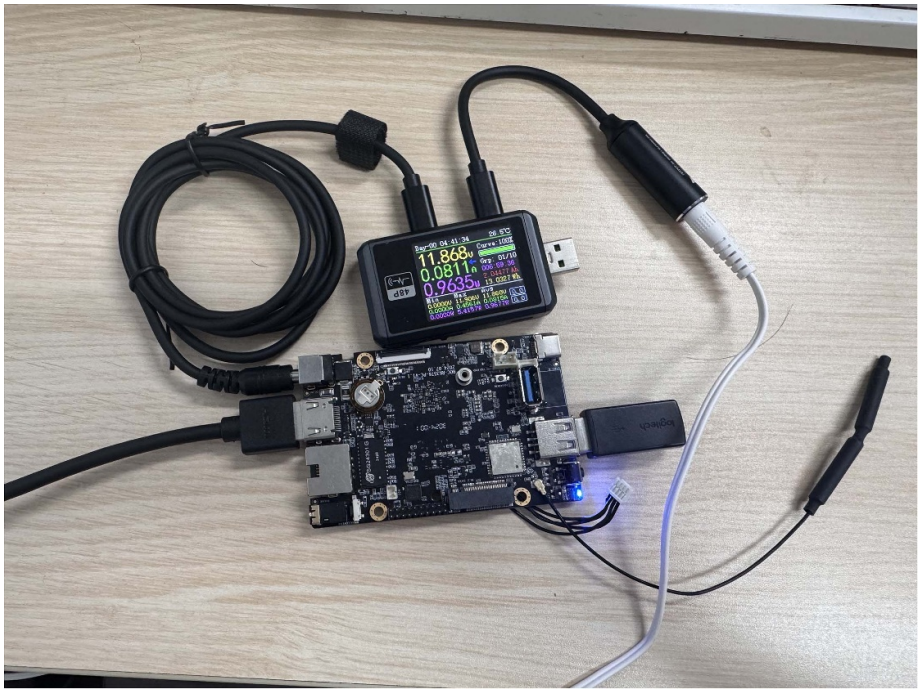


Figure 7. RK3576 power-measurement setup.

Table 6 shows the change in accuracy after INT8 quantisation on the RK3576 platform. YOLOv8s loses 0.8 percentage points after INT8 quantisation, RAD-YOLO loses 1.1 percentage points and RAD-YOLO-Slim loses 1.0 percentage point. Both FP32 and INT8 results are reported to distinguish model accuracy from deployment accuracy.

Table 7 and figure 7 report power consumption and energy efficiency. Energy efficiency is calculated as FPS divided by average power. The idle system consumes 0.96 W on average. YOLOv8s consumes 4.25 W and reaches 8.24 FPS W^{-1}. RAD-YOLO consumes 5.38 W and reaches 4.09 FPS W^{-1} because of its higher computational cost. RAD-YOLO-Slim consumes 4.32 W, runs at 51 FPS and reaches 11.81 FPS W^{-1}. RAD-YOLO-Slim has the highest inference speed and energy efficiency among the three tested models. This measurement is relevant to edge deployment, but it does not establish performance during a long-duration UAV mission.

Additional annotation-free tests on self-collected urban street and campus scenes provide supplementary qualitative examples, as shown in figures 8 and 9 and summarised in table 8. Because these images do not have ground-truth annotations, average confidence and the number of detections are used only as auxiliary indicators and should not be interpreted as standard accuracy metrics. In the urban street scene, RAD-YOLO-Slim increases average confidence from 0.65 to 0.78 and detects 27 objects compared with 15 for YOLOv8s. In the campus scene, it increases average confidence from 0.68 to 0.81 and detects 35 objects compared with 12 for YOLOv8s. The qualitative examples comprise distant vehicles, partially

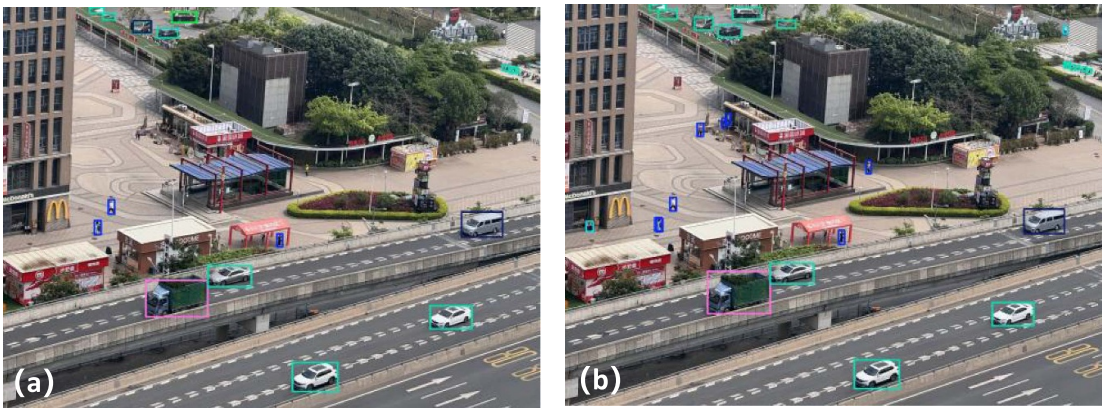


Figure 8. Detection comparison in a self-collected urban street scene.

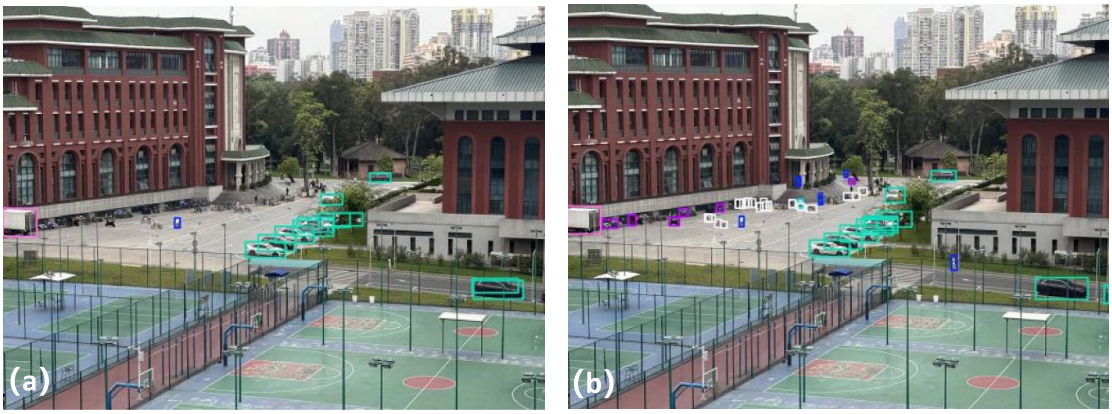


Figure 9. Detection comparison in a self-collected campus scene.

Table 8. Annotation-free evaluation on self-collected scenes.

Model	Scene type	Average detection confidence	Number of detections
YOLOv8s	Urban street	0.65	15
RAD-YOLO-Slim	Urban street	0.78	27
YOLOv8s	Campus scene	0.68	12
RAD-YOLO-Slim	Campus scene	0.81	35

occluded pedestrians and closely spaced bicycles. As these images lack ground-truth annotations, they are used only for qualitative inspection and are excluded from quantitative accuracy assessment.

6. Discussion and limitations

The ablation results suggest that RAD-YOLO benefits from feature enhancement, gated fusion and localisation optimisation. AFCBAM targets weak small-object features before repeated downsampling. G-BiAFPN targets the use of shallow spatial detail during cross-scale fusion. D-WIoU and Soft-NMS address localisation and suppression in dense scenes. The progressive results are consistent with combined and complementary contributions.

The comparison between RAD-YOLO and RAD-YOLO-Slim shows the accuracy-efficiency trade-off required for UAV remote-sensing applications. RAD-YOLO may be more appropriate when detection accuracy is the primary objective or when sufficient computing resources are available. RAD-YOLO-Slim reduces parameter count and computation substantially. In the FP32 evaluation, it achieves 44.1% mAP@0.5, higher than YOLOv8s, with 5.42 M parameters and 6.2 GFLOPs. After INT8 deployment on

RK3576, it achieves 43.1% mAP@0.5 and 51 FPS under the tested setup. For traffic monitoring, inspection and emergency response, this balance among frame rate, energy consumption and detection quality may be more practical than maximising mAP alone. The evidence is limited to the VisDrone2019-DET split and the comparison models listed in this paper. Additional public datasets and matched recent baselines can be considered in a subsequent study using a harmonised evaluation protocol.

7. Conclusion

We propose RAD-YOLO for small-object detection in UAV remote-sensing imagery and develop RAD-YOLO-Slim for edge deployment. RAD-YOLO improves YOLOv8s through AFCBAM feature enhancement, G-BiAFPN gated cross-scale fusion, D-WIoU localisation optimisation and Soft-NMS dense-object post-processing. On VisDrone2019-DET, RAD-YOLO achieves 47.6% mAP@0.5, which is 8.6 percentage points higher than YOLOv8s. The FP32 RAD-YOLO-Slim configuration reaches 44.1% mAP@0.5 with 5.42 M parameters and 6.2 GFLOPs before INT8 deployment.

After INT8 deployment on the RK3576 platform, RAD-YOLO-Slim achieves 43.1% mAP@0.5, 51 FPS and 11.81 FPS/W. These results indicate that the model can support real-time inference on the tested platform. Evaluation on additional datasets, adverse-weather scenes and other sensing or compression settings remains necessary.

Data availability statement

The VisDrone2019-DET dataset used in this study is publicly available. The self-collected aerial images used for qualitative evaluation are available from the corresponding author upon reasonable request.

Funding

This work was supported by the Guangdong Basic and Applied Basic Research Foundation (No. 2024A1515012293).

References

Bodla N, Singh B, Chellappa R and Davis L S 2017 Soft-NMS: improving object detection with one line of code *Proc. IEEE Int. Conf. on Computer Vision (ICCV)* pp 5561–9

Dong Y, Zeng H and Hou S 2025 LMUAV-YOLOv8: lightweight network for object detection in low-altitude UAV vision *Comput. Eng. Appl.* **61** 94–110

Ge Z, Liu S, Wang F, Li Z and Sun J 2021 YOLOX: exceeding YOLO Series in 2021 (arXiv:2107.08430)

Howard A G *et al* 2017 MobileNets: efficient convolutional neural networks for mobile vision applications (arXiv:1704.04861)

Hu J, Shen L and Sun G 2018 Squeeze-and-excitation networks *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 7132–41

Jacob B *et al* 2018 Quantization and training of neural networks for efficient integer-arithmetic-only inference *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 2704–13

Jocher G, Chaurasia A and Qiu J 2023 Ultralytics YOLOv8 *GitHub repository* (avaliable at: https://github.com/ultralytics/ultralytics) (Accessed 11 June 2026)

Lee J, Park S, Mo S, Ahn S and Shin J 2021 Layer-adaptive sparsity for the magnitude-based pruning (arXiv:2010.07611)

Lin T Y, Goyal P, Girshick R, He K and Dollar P 2017 Focal loss for dense object detection *Proc. IEEE Int. Conf. on Computer Vision (ICCV)* pp 2980–8

Liu S, Huang D and Wang Y 2019 Learning spatial fusion for single-shot object detection (arXiv:1911.09516)

Liu T *et al* 2024 Improved YOLOv5s UAV view small target detection algorithm *Comput. Eng. Appl.* **60** 110–21

Liu W *et al* 2016 SSD: single shot multibox detector *European Conf. on Computer Vision (ECCV)* (Springer) pp 21–37

Redmon J, Divvala S, Girshick R and Farhadi A. 2016 You only look once: unified, real-time object detection *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 779–88

Ren S, He K, Girshick R and Sun J 2015 Faster R-CNN: towards real-time object detection with region proposal networks *Advances in Neural Information Processing Systems* vol 28 pp 91–99

Tan M, Pang R and Le Q V 2020 EfficientDet: scalable and efficient object detection *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 10781–90

Tong K and Wu Y 2024 Research advances on deep learning based small object detection benchmarks *Acta Electron. Sin.* **52** 1016–40

Tong Z, Chen Y, Xu Z and Yu R 2023 Wise-IoU: bounding box regression loss with dynamic focusing mechanism (arXiv:2301.10051)

Wang C Y, Yeh I H and Liao H Y M 2024 YOLOv9: learning what you want to learn using programmable gradient information (arXiv:2402.13616)

Wang G, Chen Y, An P, Hong H, Hu J and Huang T 2023 UAV-YOLOv8: a small-object-detection model based on improved YOLOv8 for UAV aerial photography scenarios *Sensors* **23** 7190
Wang J *et al* 2021 Tiny object detection in aerial images *Proc. Int. Conf. on Pattern Recognition (ICPR)* vol 42 pp 3791–8
Woo S, Park J, Lee J Y and Kweon I S. 2018 CBAM: convolutional block attention module *Proc. European Conf. on Computer Vision (ECCV)* vol 79 pp 3–19
Wu Y and Tong K 2025 Research advances on deep learning-based small object detection in UAV aerial images *Acta Aeronaut. Astronaut. Sin.* **46** 30848
Yin R, Zhao W, Fan X and Yin Y 2020 AF-SSD: an accurate and fast single shot detector for high spatial remote sensing imagery *Sensors* **20** 6530
Yue M, Zhang L, Huang J and Zhang H 2024 Lightweight and efficient tiny-object detection based on improved YOLOv8n for UAV aerial images *Drones* **8** 276
Zhang X, Zhou X, Lin M and Sun J 2018 ShuffleNet: an extremely efficient convolutional neural network for mobile devices *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* pp 6848–56
Zhang Y F, Ren W, Zhang Z, Jia Z, Wang L and Tan T 2022 Focal and efficient IoU loss for accurate bounding box regression *Neurocomputing* **506** 146–57
Zhang Z 2023 Drone-YOLO: an efficient neural network method for target detection in drone images *Drones* **7** 526
Zhang Z, Du L, Hou Y, HAO Z and Yin J 2024 Multi-feature fusion object detection algorithm for UAV images under cross-layer attention interaction *Opt. Precis. Eng.* **32** 3616–31
Zhou X, Wang D and Krahenbuhl P 2019 Objects as points (arXiv:1904.07850)
Zhu P, Wen L, Bian X, Ling H and Hu Q. 2018 Vision meets drones: a challenge (arXiv:1804.07437)
Zhu X, Lyu S, Wang X and Zhao Q. 2021 TPH-YOLOv5: improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios *Proc. IEEE/CVF Int. Conf. on Computer Vision Workshops (ICCVW)* pp 2778–88